

Introduction to DBMS

1. Introduction to SQL

Lab 1 : Create a new database named school_db and a table called students with the following columns: student_id, student_name, age, class, and address.

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_name VARCHAR(50),  
    age INT,  
    class VARCHAR(20),  
    address VARCHAR(100)  
);
```

Lab 2 : Insert five records into the students table and retrieve all records using the SELECT statement.

```
INSERT INTO students (student_name, age, class, address) VALUES  
( 'Parth', 21, '7th', 'Ahmedabad'),  
( 'Neer', 22, '6th', 'Surat'),  
( 'Nirav', 23, '8th', 'Baroda'),  
( 'Yash', 30, '5th', 'Rajkot'),  
( 'Aayush', 24, '9th', 'Ahmedabad');
```

```
SELECT * FROM students;
```

2. SQL Syntax

Lab 1: Write SQL queries to retrieve specific columns (student_name and age) from the students table.

```
SELECT student_name, age FROM students;
```

Lab 2: Write SQL queries to retrieve all students whose age is greater than 10.

```
SELECT * FROM students WHERE age > 10;
```

3. SQL Constraints

```
CREATE TABLE teachers(teacher_id INT PRIMARY KEY  
AUTO_INCREMENT,teacher_name VARCHAR(50)NOT NULL, subject  
VARCHAR(50) NOT NULL,email VARCHAR(50) UNIQUE KEY)
```

```
CREATE TABLE teachers(  
    teacher_id INT PRIMARY KEY AUTO_INCREMENT,  
    teacher_name VARCHAR(50) NOT NULL,  
    subject VARCHAR(50) NOT NULL,  
    email VARCHAR(50) UNIQUE  
);
```

Lab 2: Implement a FOREIGN KEY constraint to relate the teacher_id from the teachers table with the students table.

```
ALTER TABLE students  
ADD teacher_id INT,  
ADD FOREIGN KEY (teacher_id) REFERENCES  
teachers(teacher_id);
```

localhost / 127.0.0.1 / school / stu X

localhost/phpmyadmin/index.php?route=/table/sql&db=school&table=students

Server: 127.0.0.1 » Database: school » Table: students

Browse Structure SQL Search Insert Export Import Privileges Operations Tracking Triggers

Show query box

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0007 seconds.)

```
ALTER TABLE students ADD teacher_id INT, ADD FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id);
```

[Edit inline] [Edit] [Create PHP code]

Console

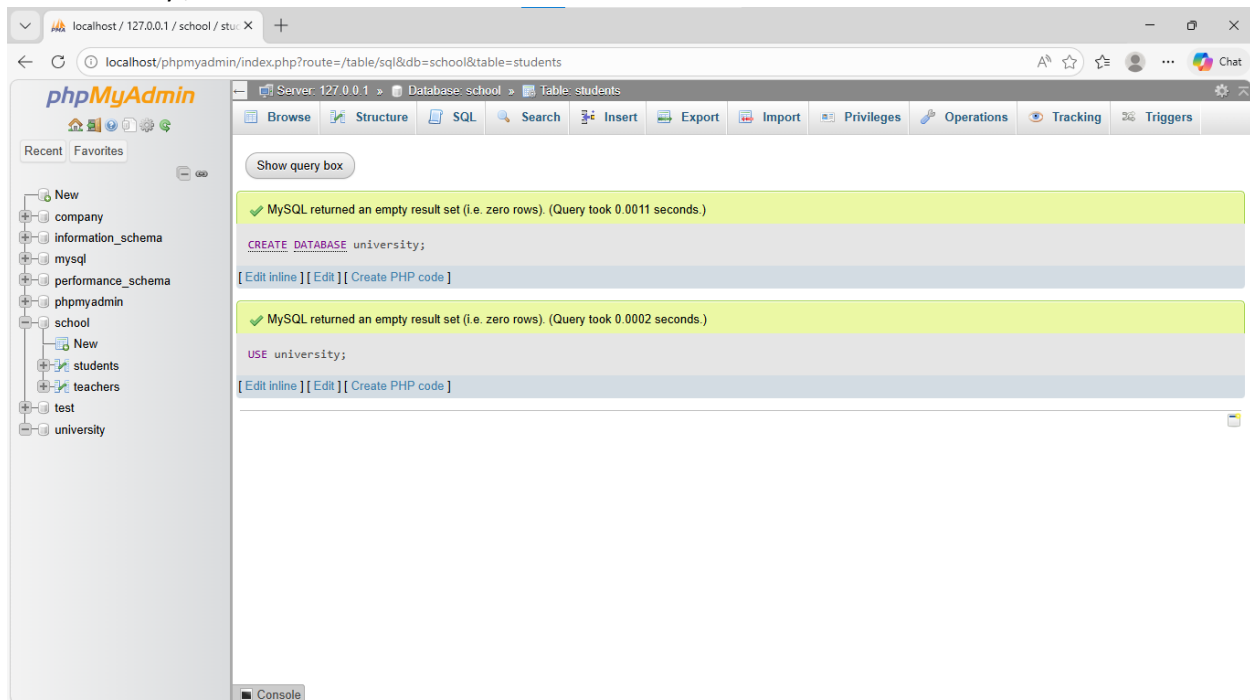
4. Main SQL Commands and Sub-commands (DDL)

Lab 1: Create a table courses with columns: course_id, course_name, and course_credits. Set the course_id as the primary key.

Lab 2: Use the CREATE command to create a database university_db.

CREATE DATABASE university;

USE university ;



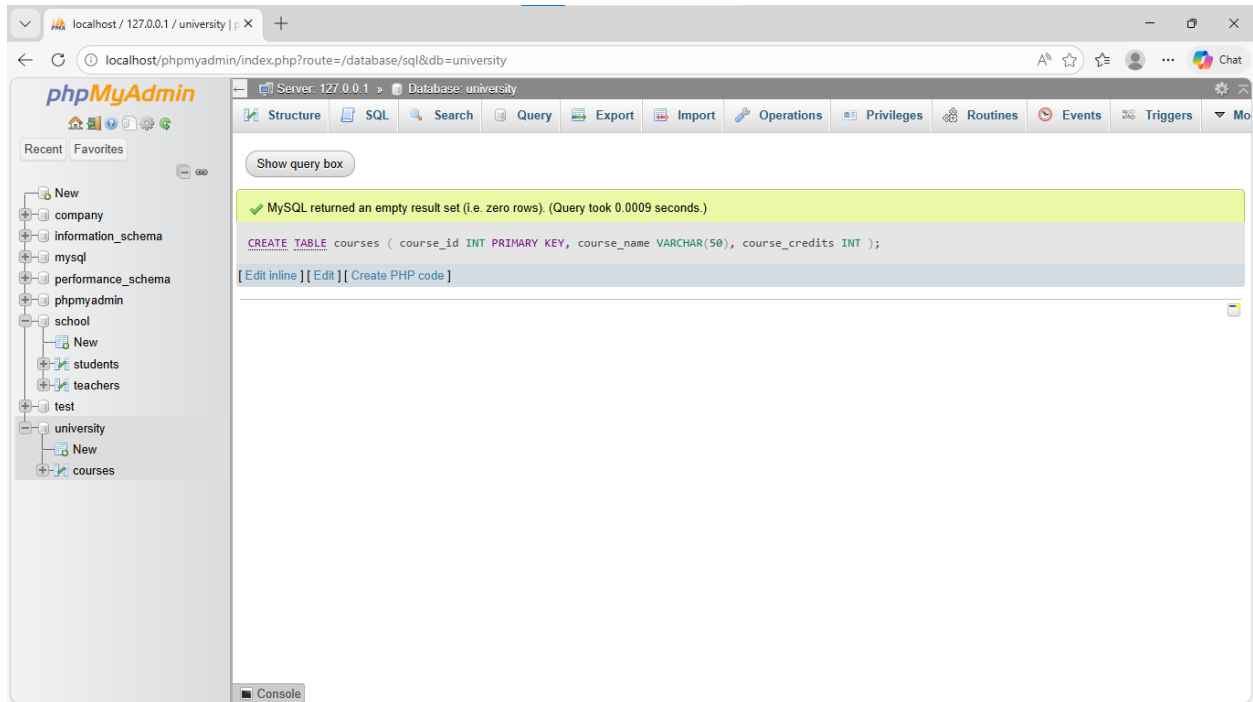
CREATE TABLE courses (

course_id INT PRIMARY KEY,

course_name VARCHAR(50),

course_credits INT

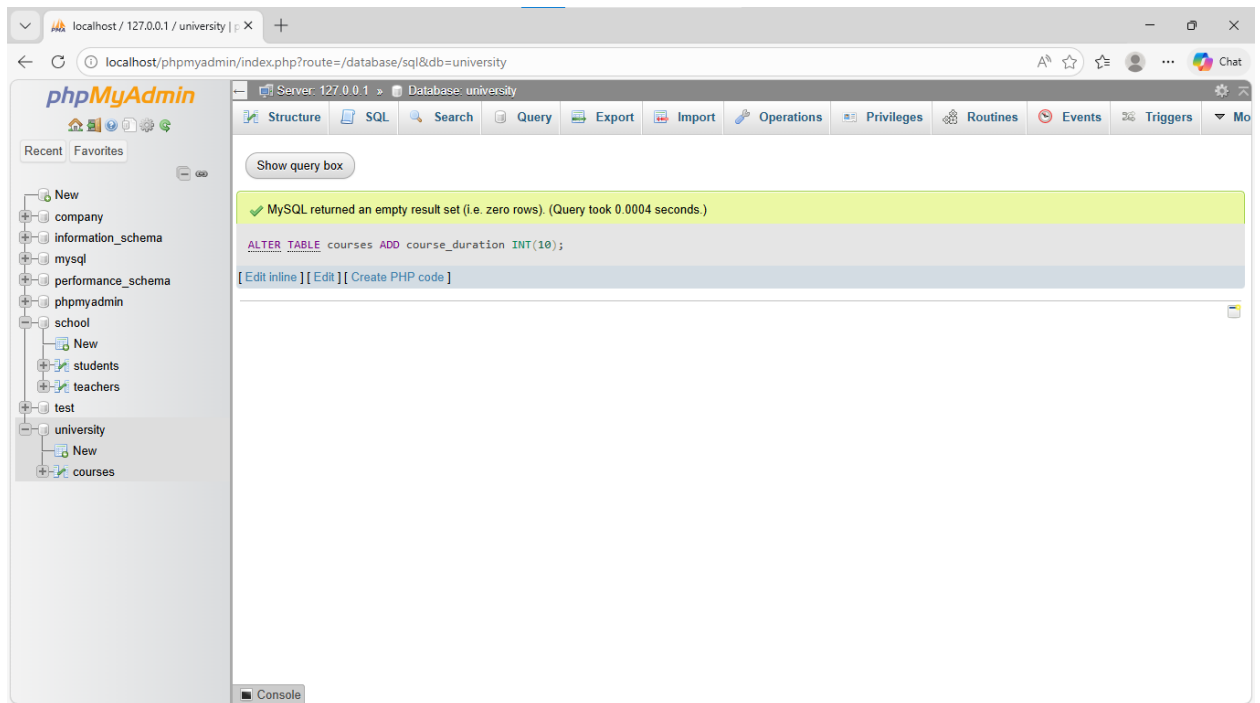
);



5.Main SQL Commands and Sub-commands (DDL)

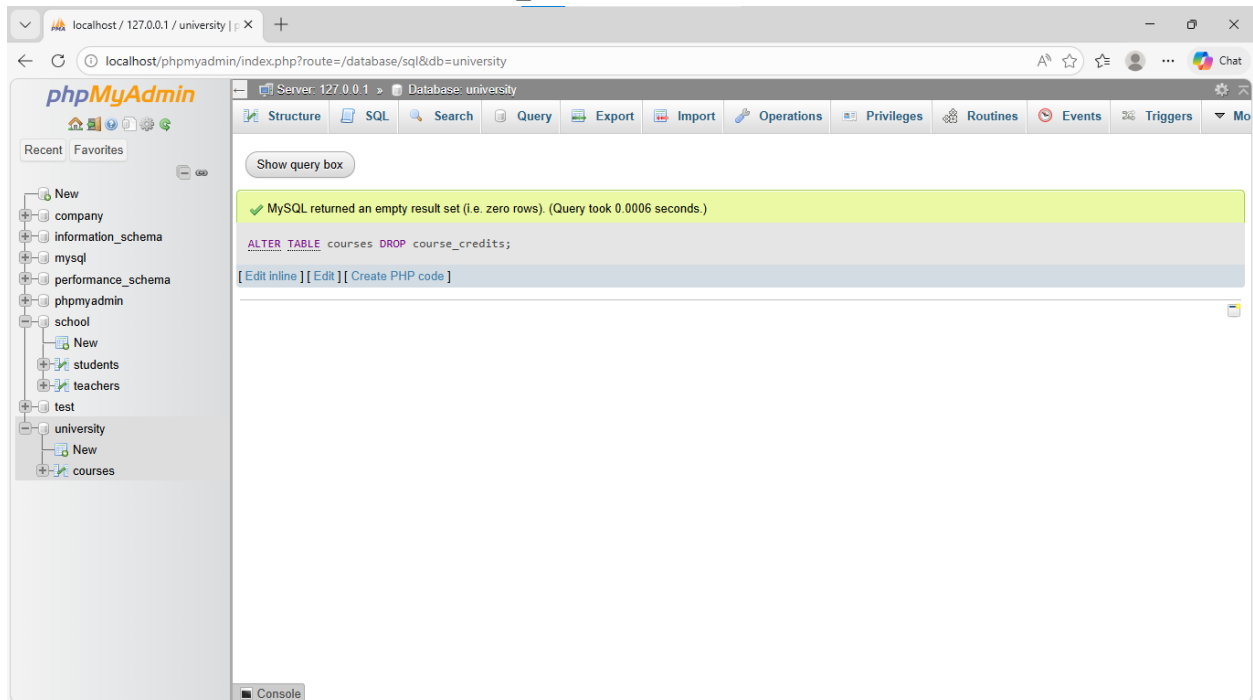
Lab 1: Modify the courses table by adding a column `course_duration` using the ALTER command.

```
ALTER TABLE courses ADD course_duration INT(10);
```



Lab 2: Drop the `course_credits` column from the `courses` table.

`ALTER TABLE courses DROP course_credits;`

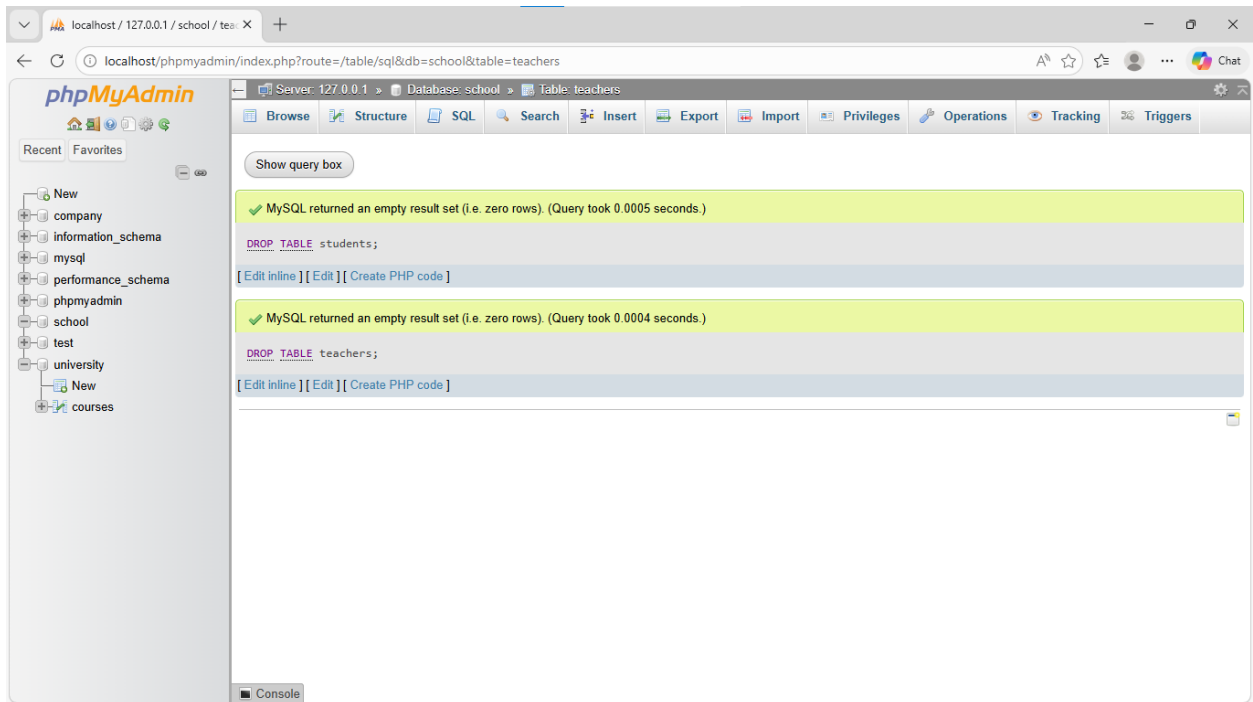


6.DROP Command

Lab 1: Drop the teachers table from the school_db database.

Lab 2: Drop the students table from the school_db database and verify that the table has been removed.

DROP TABLE students;
DROP TABLE teachers;

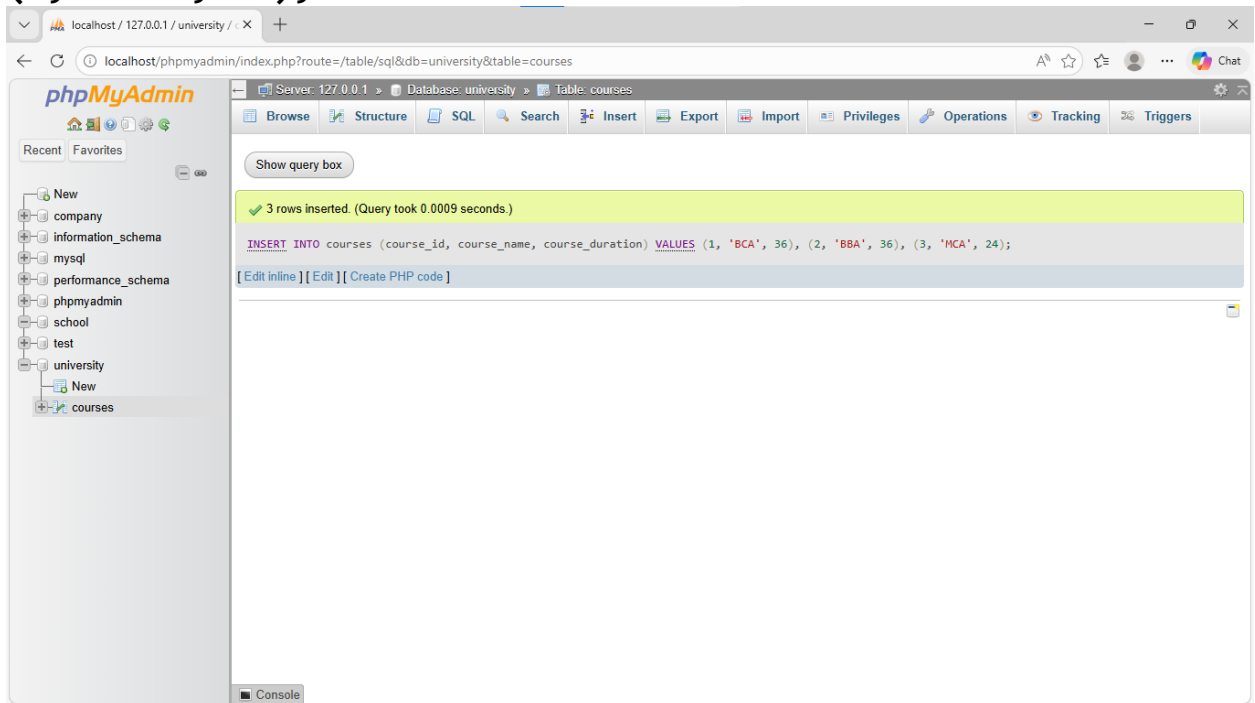


7.Data Manipulation Language (DML)

Lab 1: Insert three records into the courses table using the INSERT command.

```
INSERT INTO courses (course_id, course_name,  
course_duration) VALUES  
(1, 'BCA', 36),  
(2, 'BBA', 36),
```


(3, 'MCA', 24);

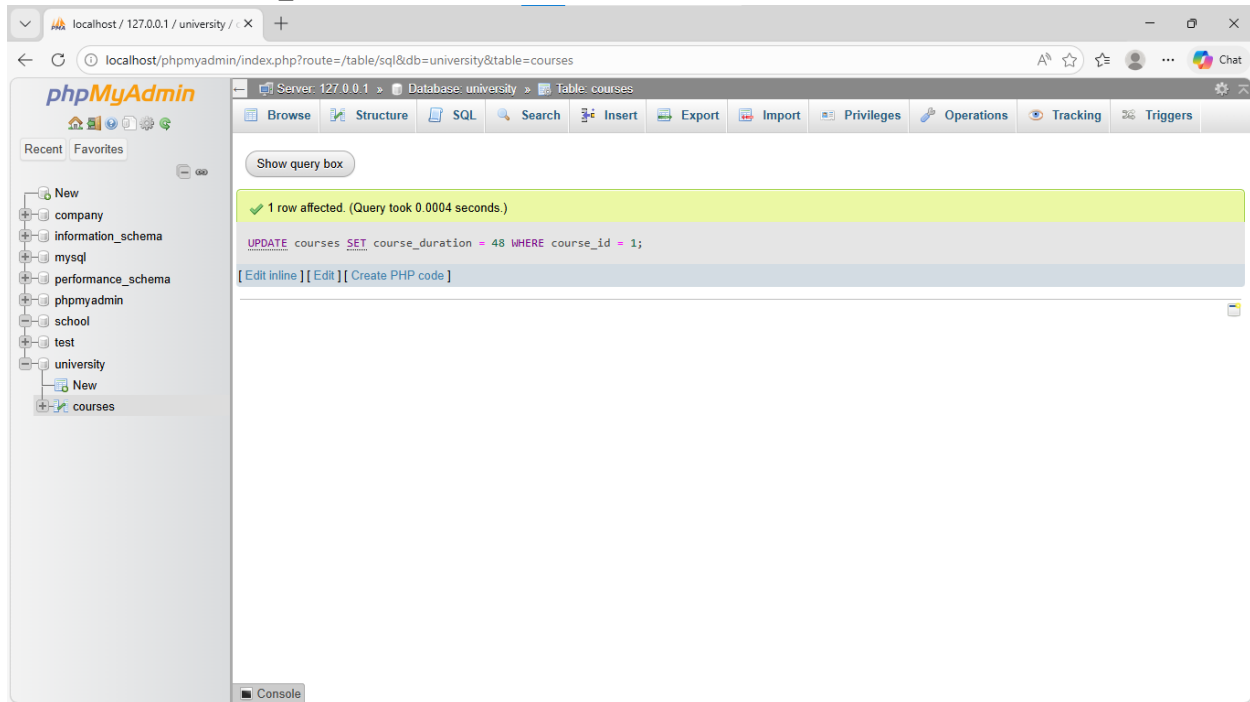


Lab 2: Update the course duration of a specific course using the UPDATE command.

UPDATE courses

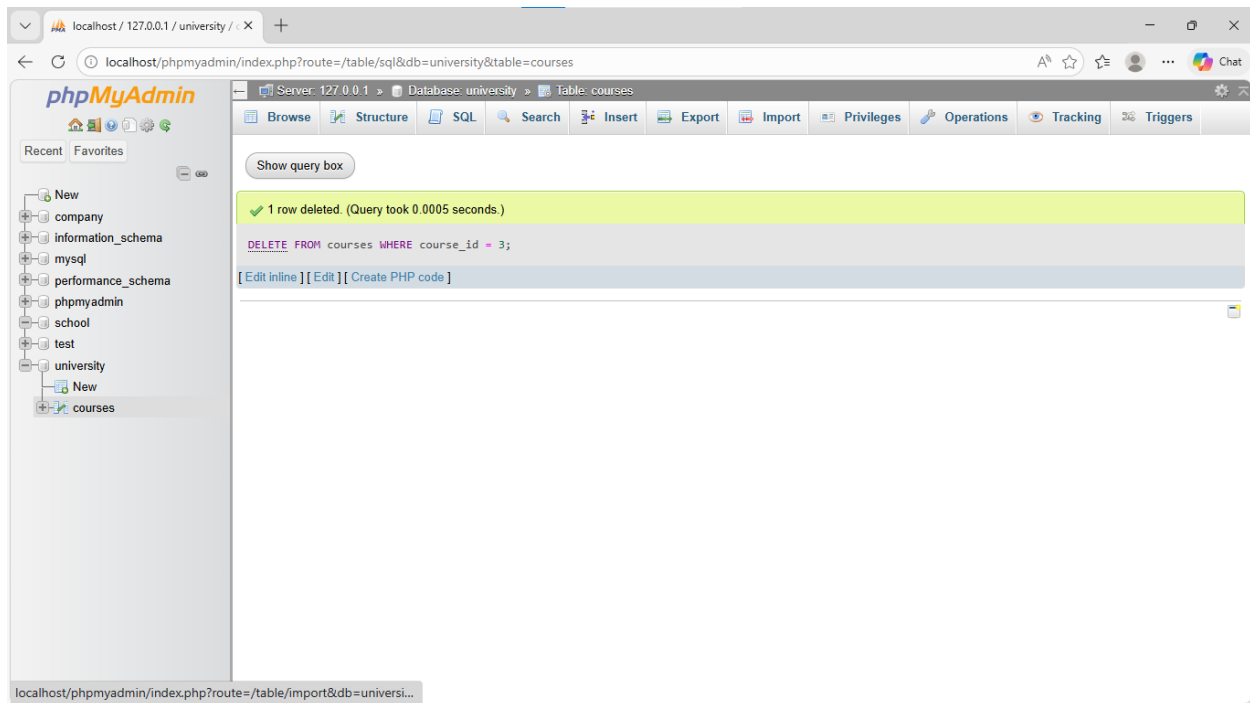
SET course_duration = 48

WHERE course_id = 1;



Lab 3: Delete a course with a specific course_id from the courses table using the DELETE command.

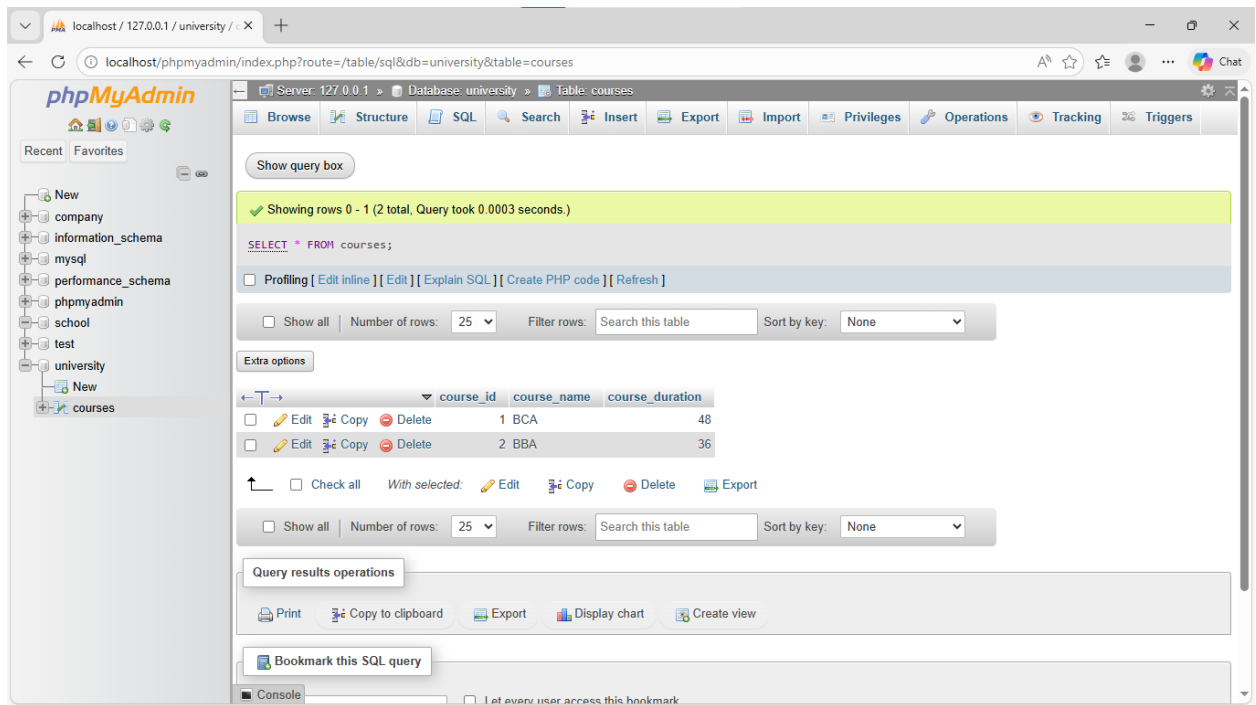
DELETE FROM courses WHERE course_id = 3;



8.Data Query Language (DQL)

Lab 1: Retrieve all courses from the courses table using the SELECT statement.

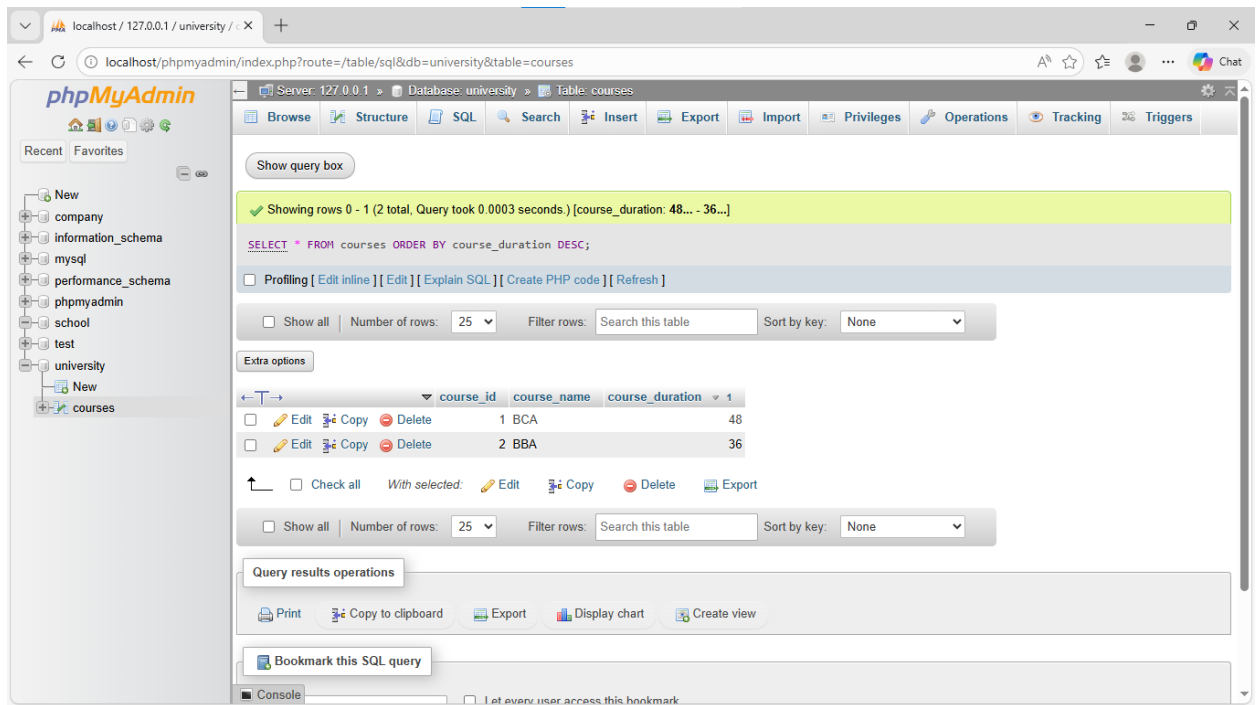
```
SELECT * FROM courses;
```



Lab 2: Sort the courses based on course_duration in descending order using ORDERBY

SELECT * FROM courses

ORDER BY course_duration DESC;

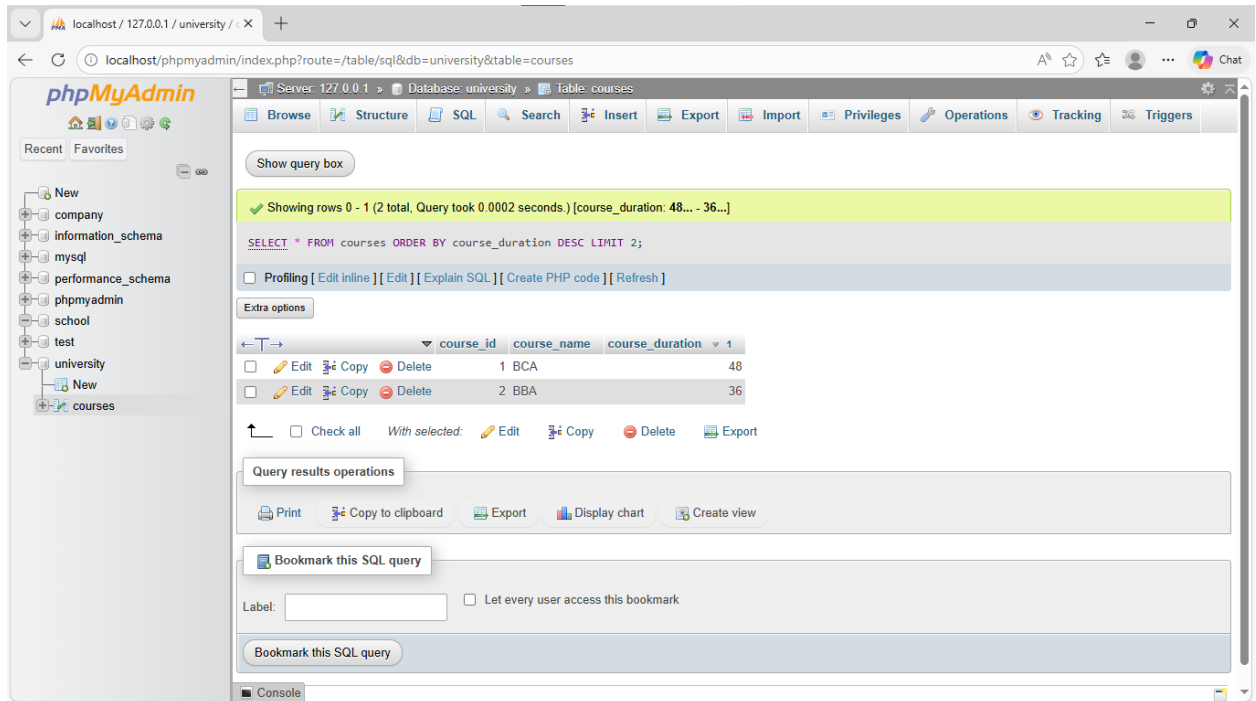


Lab 3: Limit the results of the SELECT query to show only the top two courses using LIMIT.

SELECT * FROM courses

ORDER BY course_duration DESC

LIMIT 2;



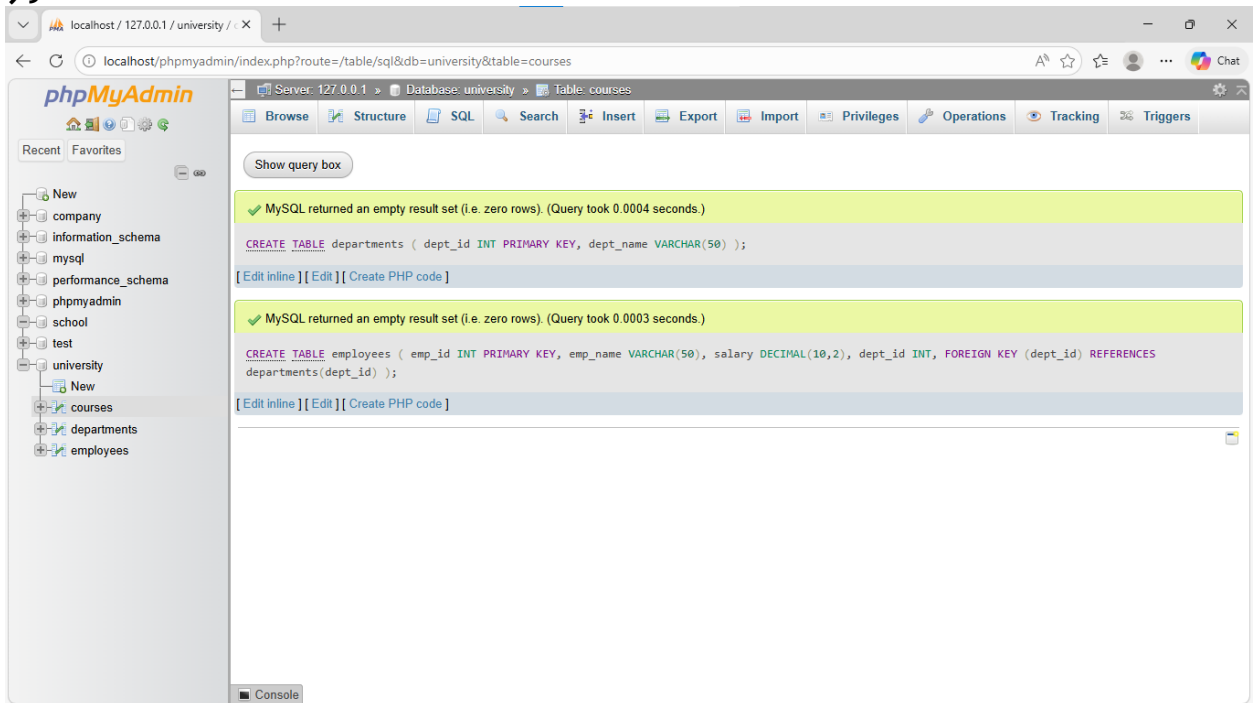
9.SQL Joins

Lab 1: Create two tables: departments and employees. Perform an INNER JOIN to display employees along with their respective departments.

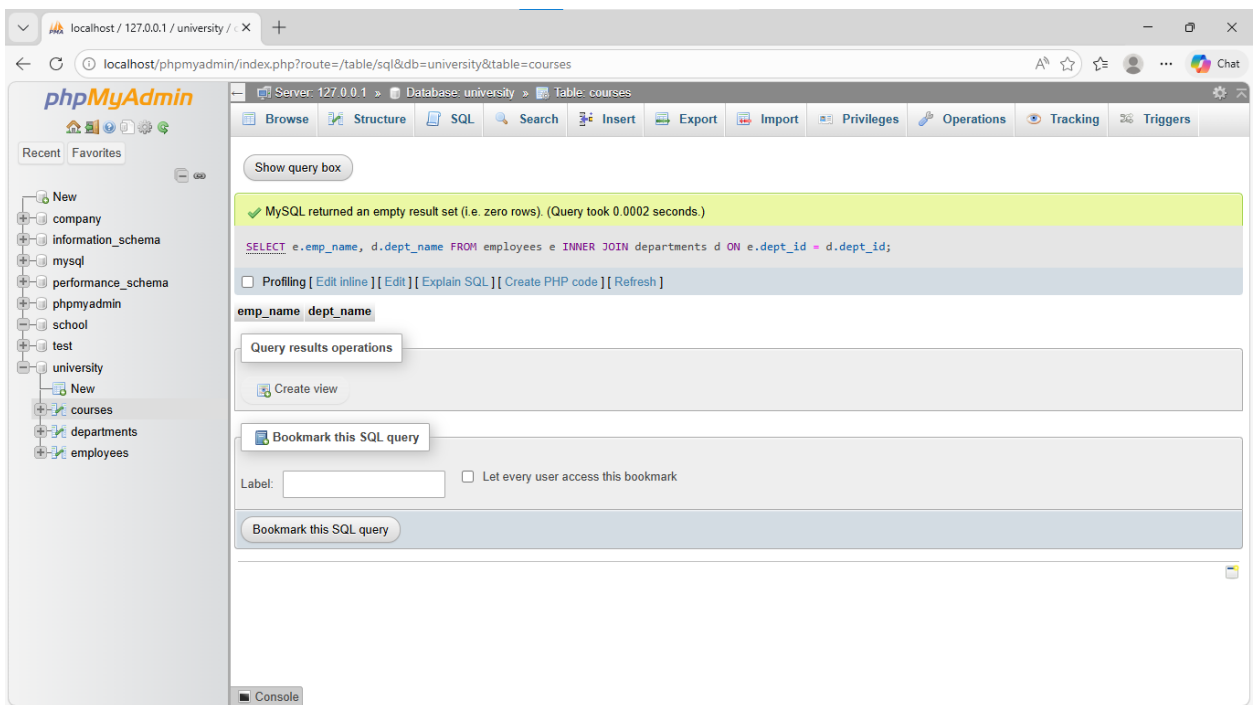
```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary DECIMAL(10,2),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES  
departments(dept_id)
```

);



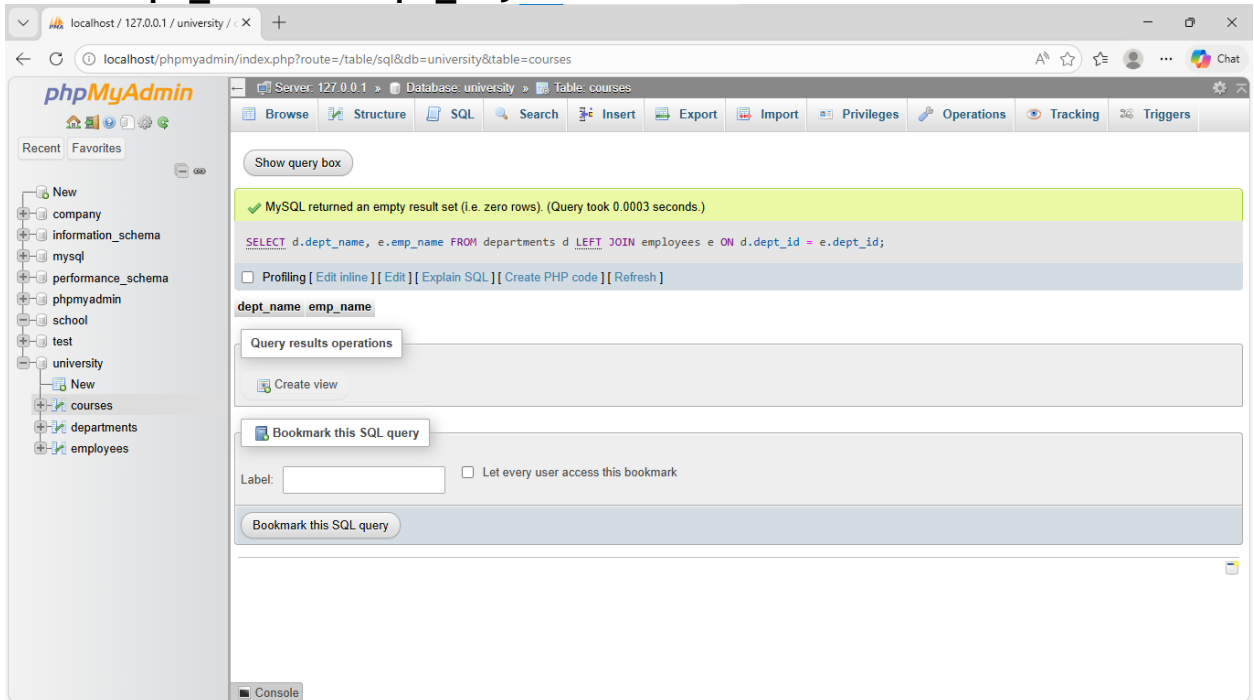
Lab 2: Use a LEFT JOIN to show all departments, even those without employees.



**SELECT e.emp_name, d.dept_name
FROM employees e**

```
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

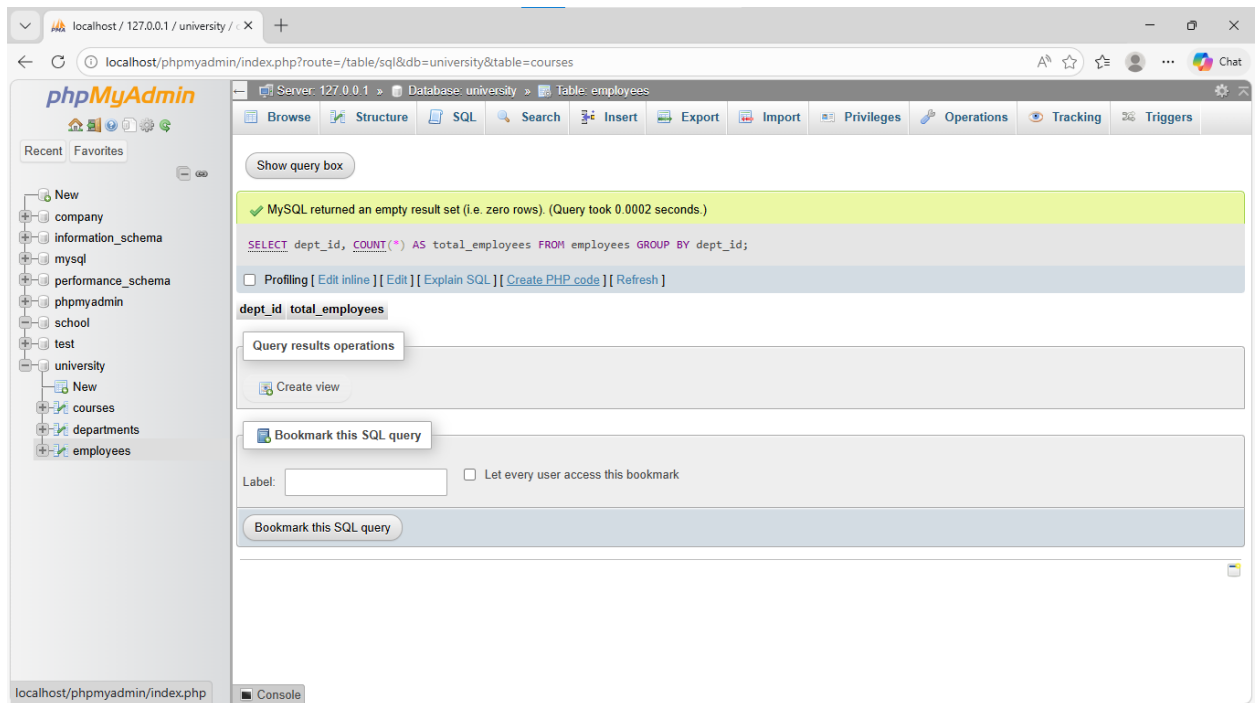
```
SELECT d.dept_name, e.emp_name
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id;
```



10.SQL Group By

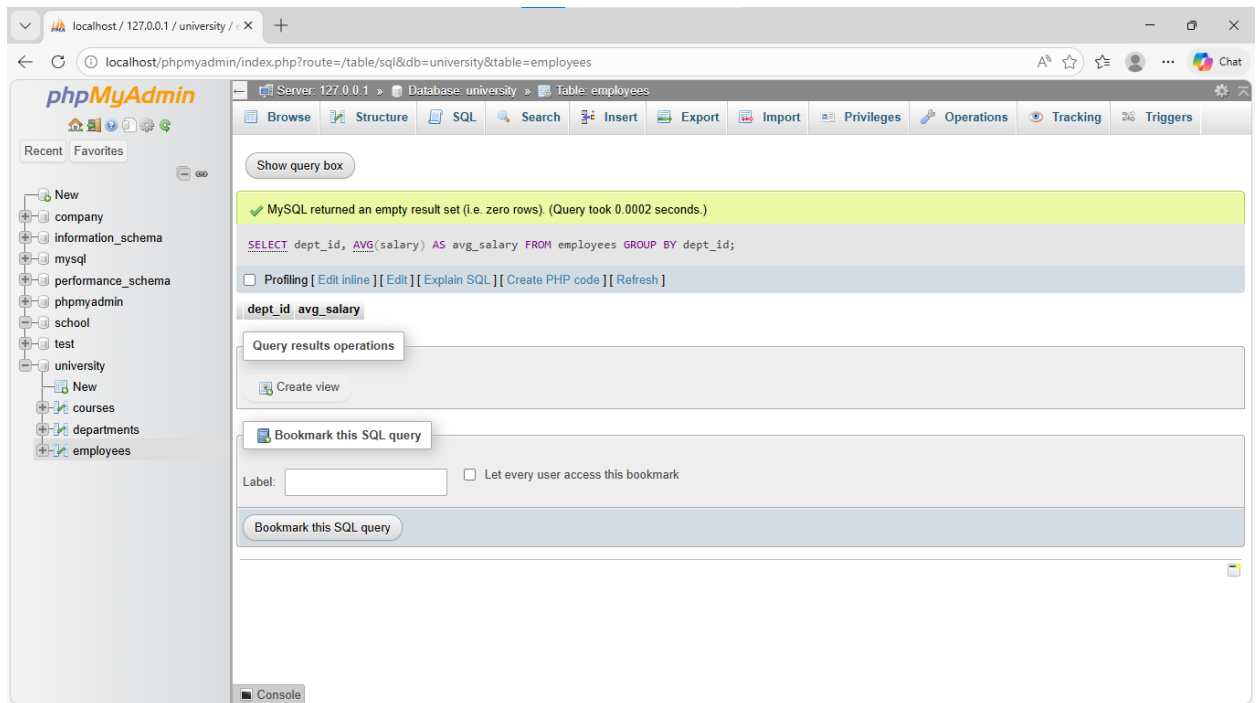
Lab 1: Group employees by department and count the number of employees in each department using GROUP BY.

```
SELECT dept_id, COUNT(*) AS total_employees
FROM employees
GROUP BY dept_id;
```

Lab 2: Use the AVG aggregate function to find the average salary of employees in each department.

```
SELECT dept_id, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY dept_id;
```



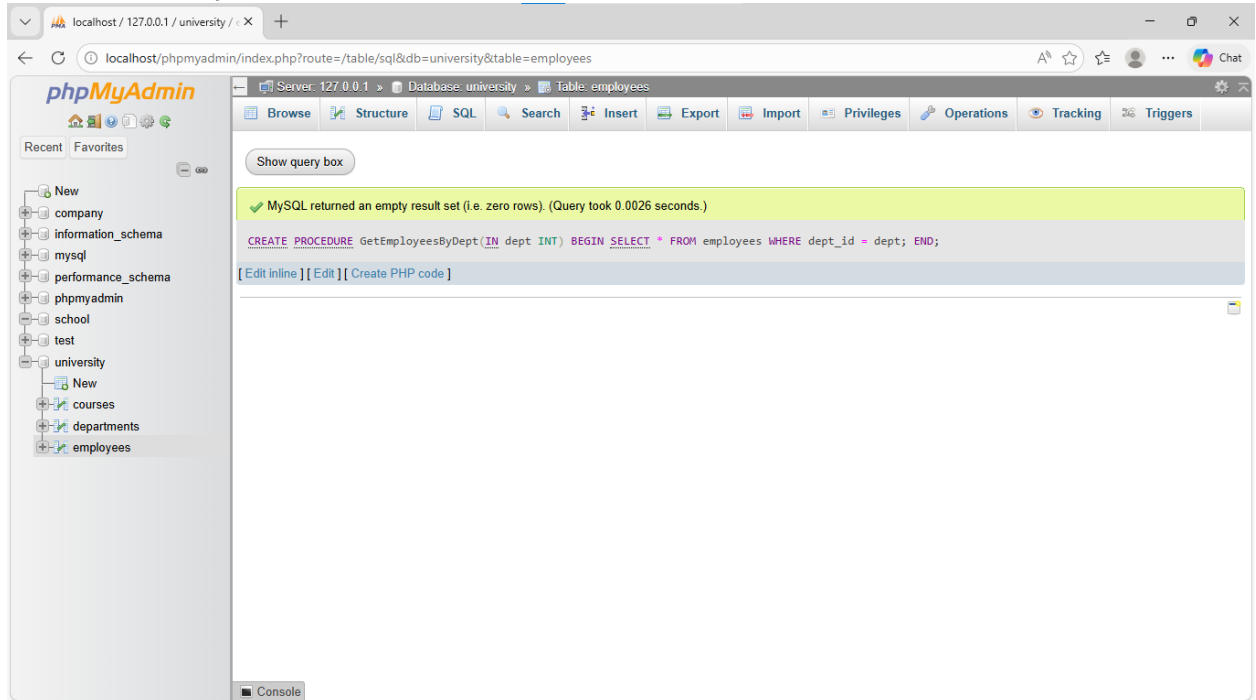
11.SQL Stored Procedure

Lab 1: Write a stored procedure to retrieve all employees from the employees table based on department.

DELIMITER //

```
CREATE PROCEDURE GetEmployeesByDept(IN dept INT)
BEGIN
    SELECT * FROM employees WHERE dept_id = dept;
END //
```

DELIMITER ;



Lab 2: Write a stored procedure that accepts `course_id` as input and returns the course details.

DELIMITER //

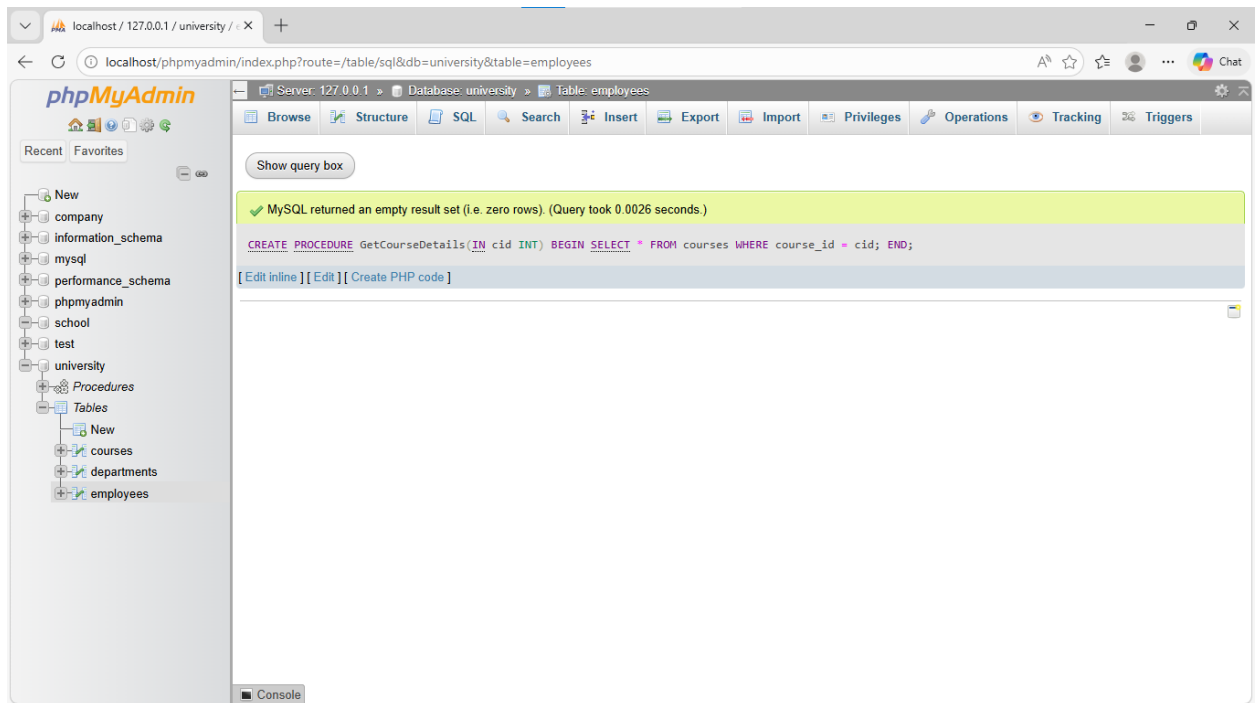
```
CREATE PROCEDURE GetCourseDetails(IN cid INT)
```

```
BEGIN
```

```
    SELECT * FROM courses WHERE course_id = cid;
```

```
END //
```

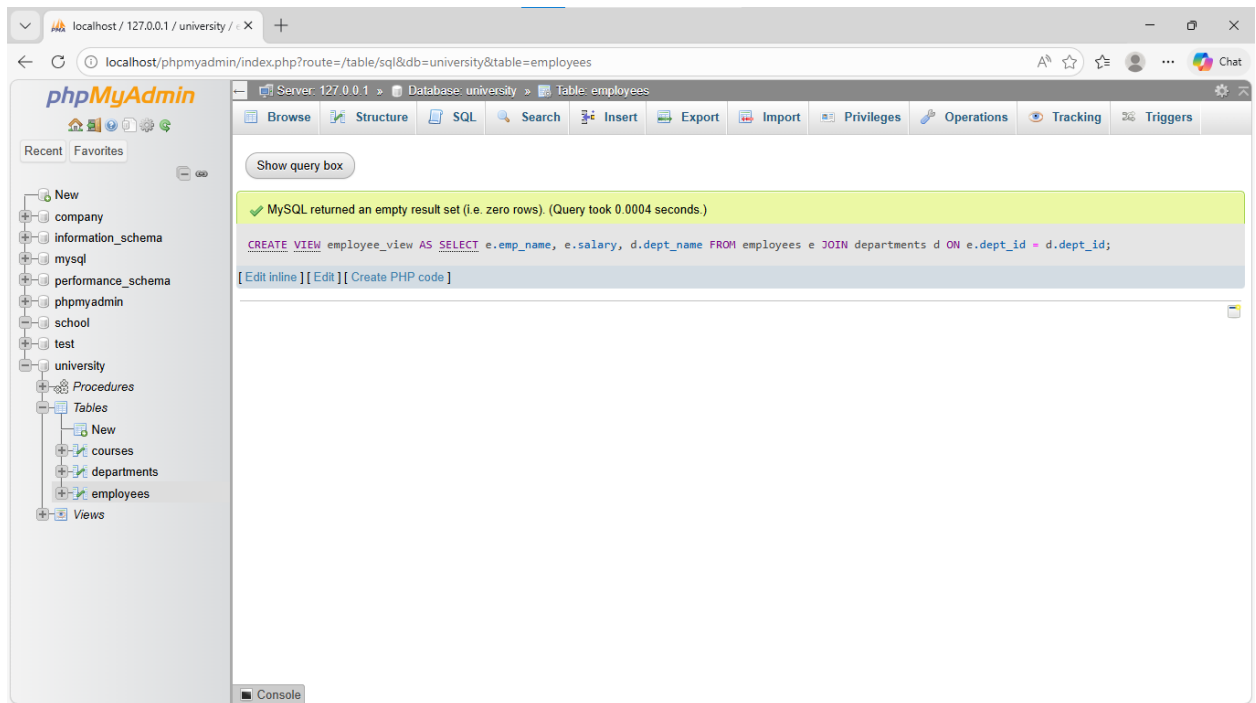
DELIMITER ;



12.SQL View

Lab 1: Create a view to show all employees along with their department names.

```
CREATE VIEW employee_view AS  
  
SELECT e.emp_name, e.salary, d.dept_name  
  
FROM employees e  
  
JOIN departments d ON e.dept_id = d.dept_id;
```



Lab 2: Modify the view to exclude employees whose salaries are below \$50,000.

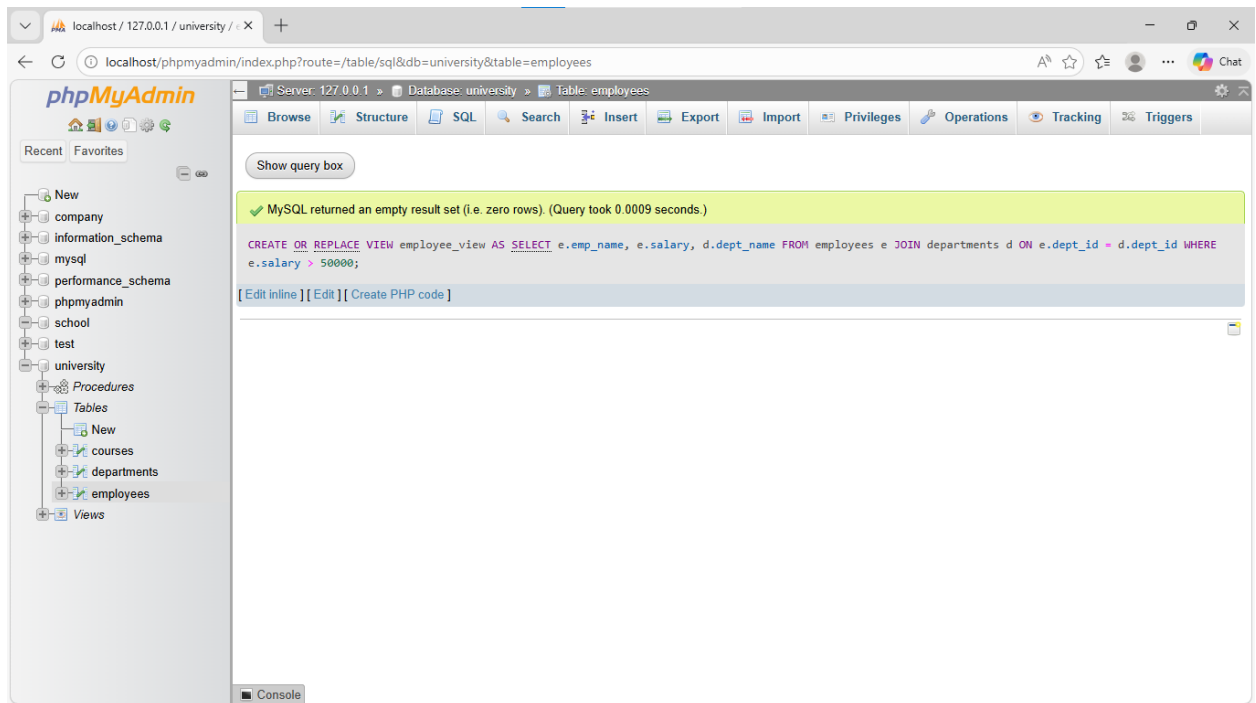
CREATE OR REPLACE VIEW employee_view AS

SELECT e.emp_name, e.salary, d.dept_name

FROM employees e

JOIN departments d ON e.dept_id = d.dept_id

WHERE e.salary > 50000;



13.SQL Triggers

Lab 1: Create a trigger to automatically log changes to the employees table when a new employee is added.

Lab 2: Create a trigger to update the last_modified timestamp whenever an employee record is updated.

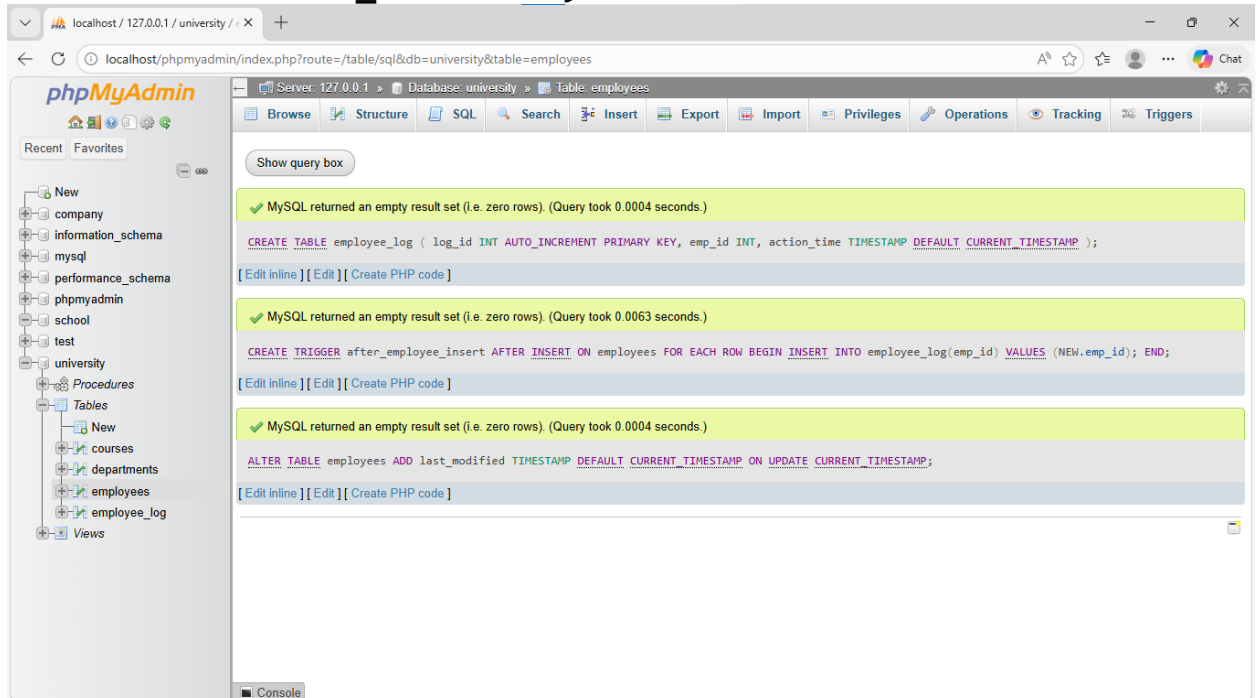
```
CREATE TABLE employee_log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT,  
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
DELIMITER //
```

```
CREATE TRIGGER after_employee_insert  
AFTER INSERT ON employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO employee_log(emp_id)
```

```
VALUES (NEW.emp_id);
END //
```

```
DELIMITER ;
ALTER TABLE employees
ADD last_modified TIMESTAMP DEFAULT CURRENT_TIMESTAMP
ON UPDATE CURRENT_TIMESTAMP;
```



EXTRA LAB PRACTISE FOR DATABASE CONCEPTS

1. Introduction to SQL
2. Lab 3: Create a database called library_db and a table books with columns: book_id, title, author, publisher, year_of_publication, and price. Insert five records into the table.

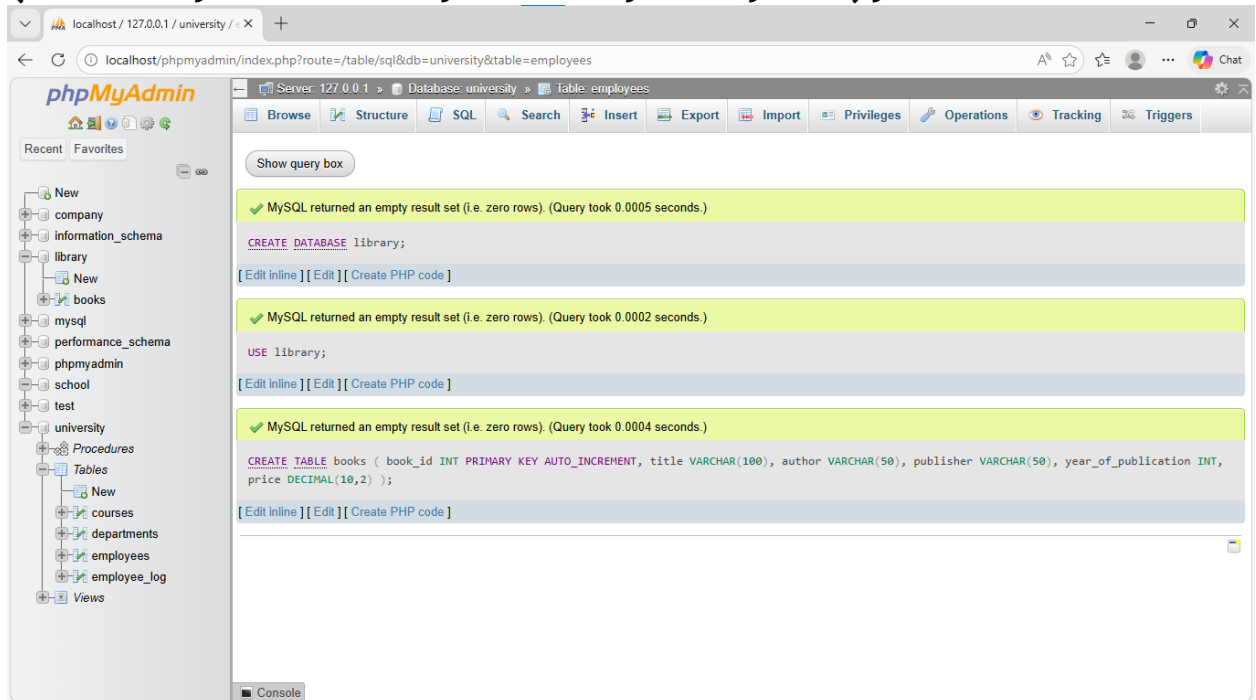
```
CREATE DATABASE library;
USE library;
```

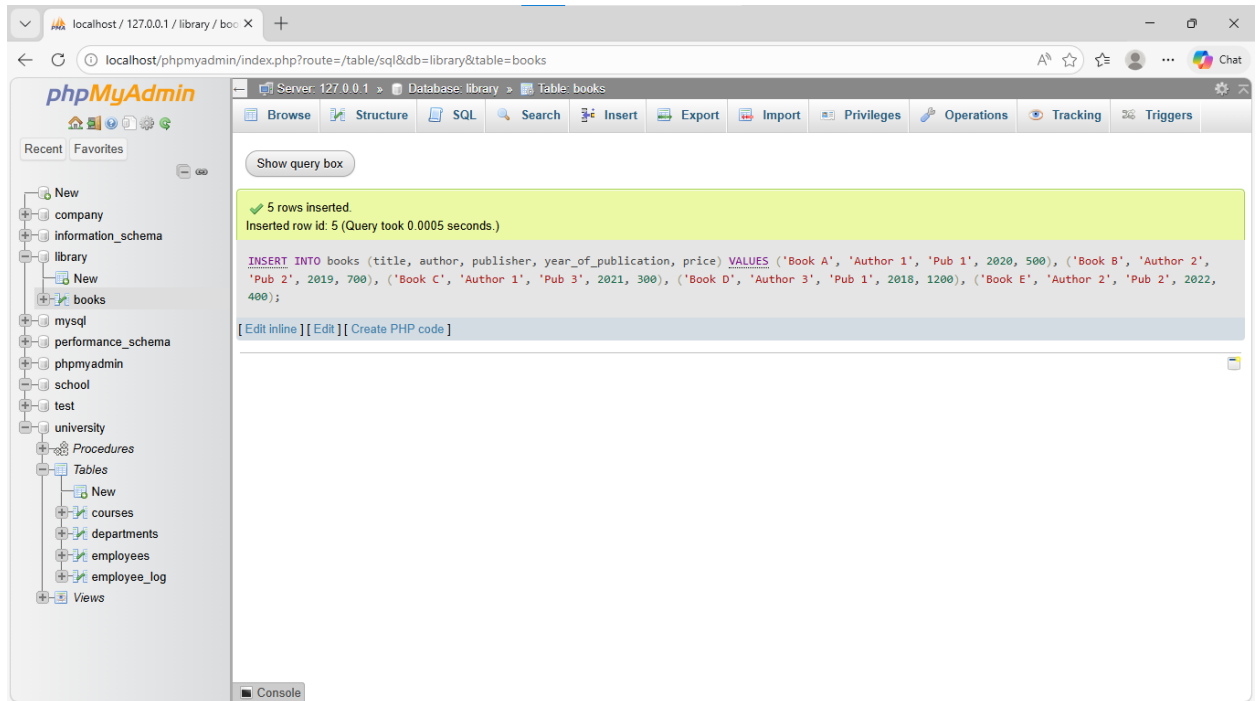
```
CREATE TABLE books (
    book_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100),
    author VARCHAR(50),
    publisher VARCHAR(50),
```

```

        year_of_publication INT,
        price DECIMAL(10,2)
    );
INSERT INTO books (title, author, publisher,
year_of_publication, price) VALUES
('Book A', 'Author 1', 'Pub 1', 2020, 500),
('Book B', 'Author 2', 'Pub 2', 2019, 700),
('Book C', 'Author 1', 'Pub 3', 2021, 300),
('Book D', 'Author 3', 'Pub 1', 2018, 1200),
('Book E', 'Author 2', 'Pub 2', 2022, 400);

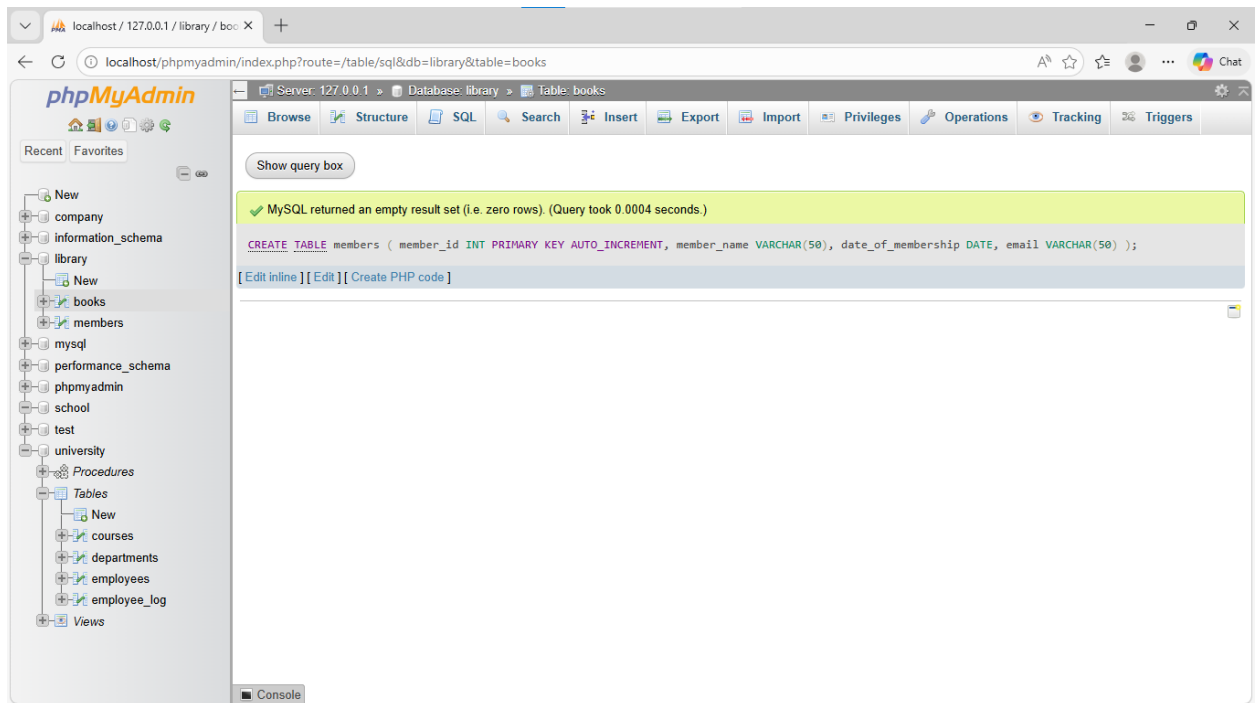
```





Lab 4: Create a table members in library_db with columns: member_id, member_name, date_of_membership, and email. Insert five records into this table.

```
CREATE TABLE members (  
    member_id INT PRIMARY KEY AUTO_INCREMENT,  
    member_name VARCHAR(50),  
    date_of_membership DATE,  
    email VARCHAR(50)  
);
```



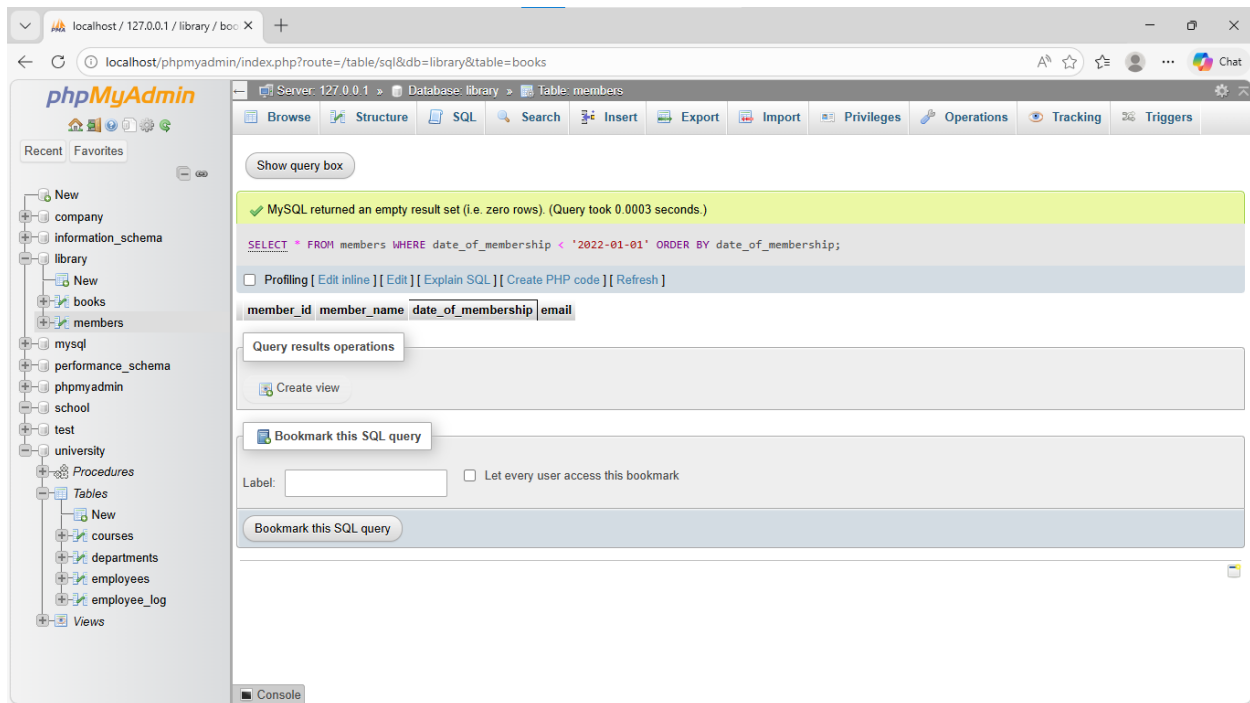
3. SQL Syntax

Lab 3: Retrieve all members who joined the library before 2022. Use appropriate SQL syntax with WHERE and ORDER BY.

SELECT * FROM members

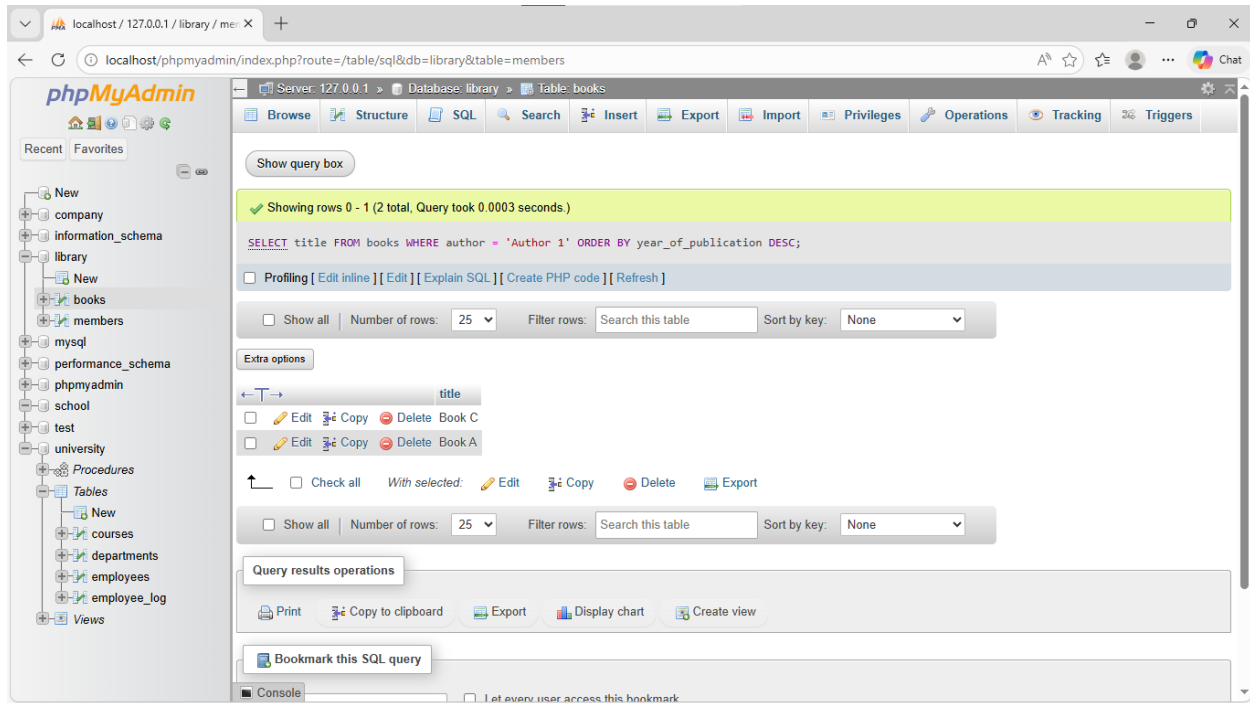
WHERE date_of_membership < '2022-01-01'

ORDER BY date_of_membership;



Lab 4: Write SQL queries to display the titles of books published by a specific author. Sort the results by year_of_publication in descending order.

```
SELECT title  
FROM books  
WHERE author = 'Author 1'  
ORDER BY year_of_publication DESC;
```

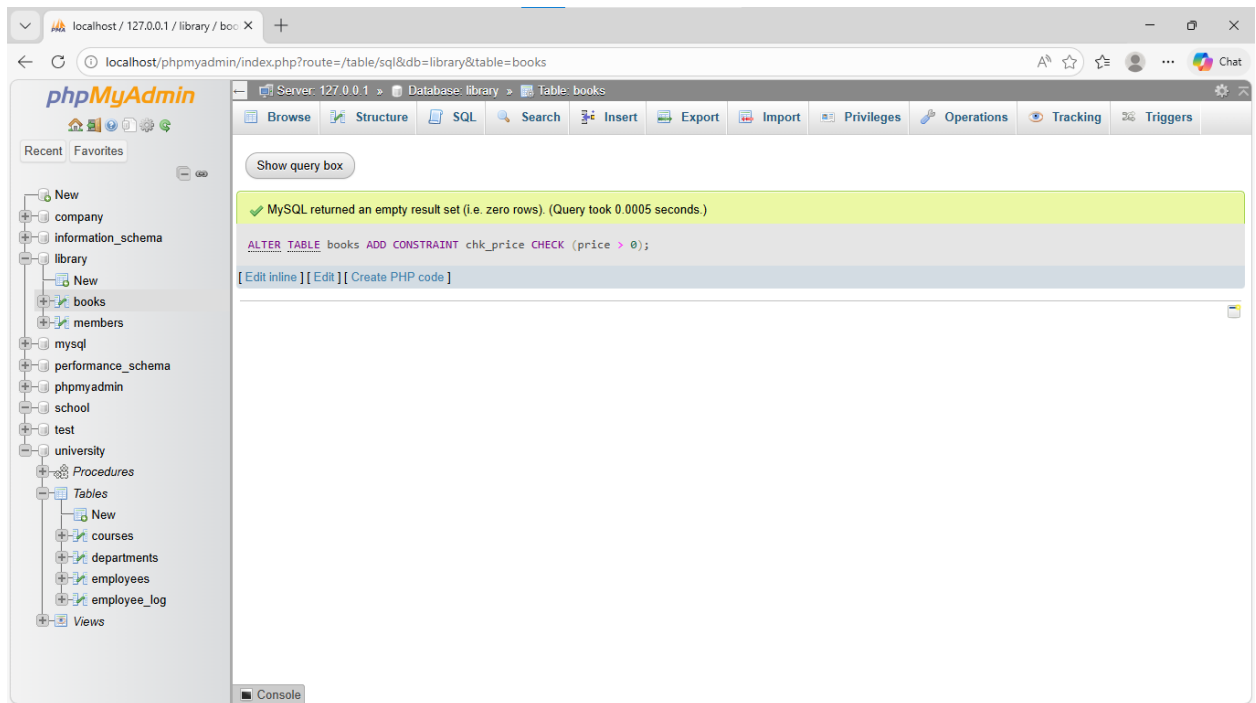


4.SQL Constraints

Lab 3: Add a CHECK constraint to ensure that the price of books in the books table is greater than 0.

ALTER TABLE books

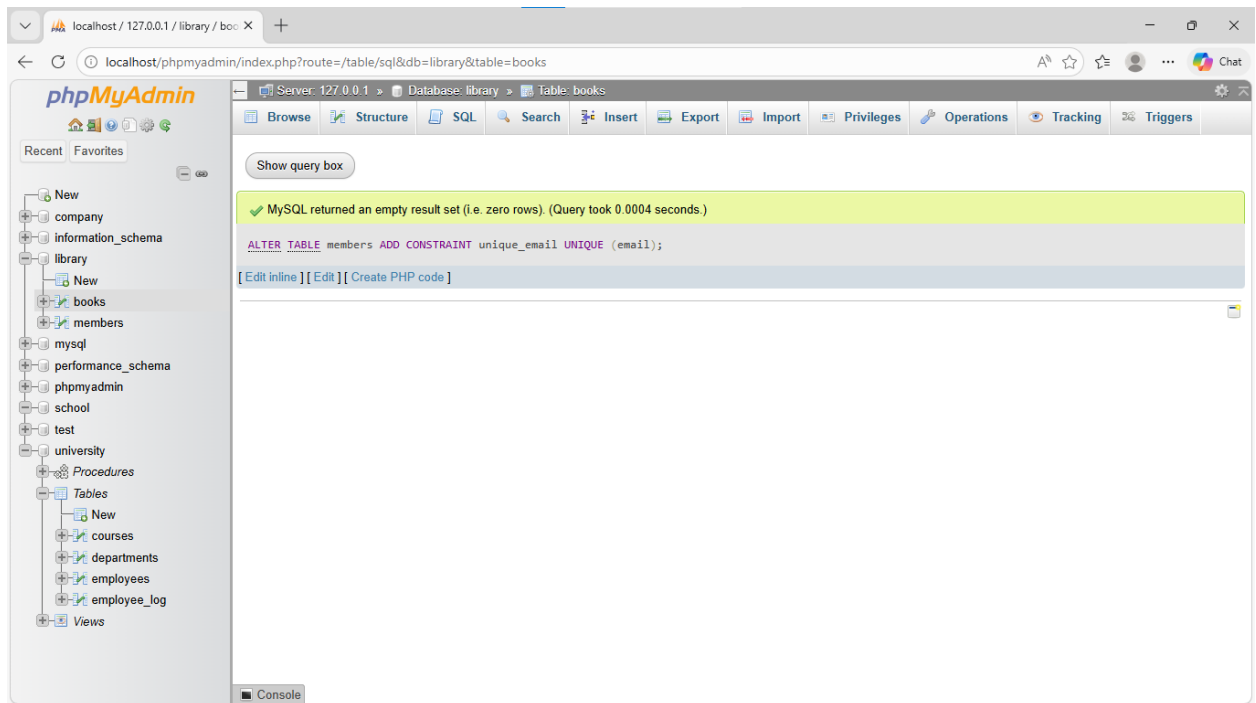
ADD CONSTRAINT chk_price CHECK (price > 0);



Lab 4: Modify the members table to add a UNIQUE constraint on the email column, ensuring that each member has a unique email address.

ALTER TABLE members

ADD CONSTRAINT unique_email UNIQUE (email);



4. Main SQL Commands and Sub-commands (DDL)

Lab 3: Create a table authors with the following columns: author_id, first_name, last_name, and country. Set author_id as the primary key.

DELETE FROM books WHERE price > 100;